
ML Inference Optimization

Joshua Wisdom Momo
jmomo@andrew.cmu.edu

Abstract

Vision-Language-Action models like SmolVLA (450M parameters) are too slow for real-time robot control. Their baseline inference is typically at 258 ms per action chunk (this is over $5\times$ the 50 ms budget for 20 Hz control). I present an architecture-informed optimization pipeline that achieves a **$68\times$ throughput improvement** (193.8 to 13,185 actions/second) on an A10G GPU while maintaining identical peak memory (0.859 GB). The approach combines three main techniques: (1) `torch.compile` with CUDA graph capture, which fuses over 12,000 (12,203 on A10G) per-call kernel launches into optimized execution graphs for a $6\times$ speedup; (2) action chunk caching, which amortizes each forward pass across 10 timesteps by reusing predicted action chunks; and (3) ODE denoising step reduction from 10 to 8 (grounded in the constant velocity property of flow matching optimal transport paths). Several attempted optimizations failed or hurt performance, including FP16 autocast on A10G (4% slowdown versus native BF16) and direct FP16 weight conversion (dtype mismatch crash). The resulting raw score of 71.2 places the submission in the “exceptional” category (>10.0), pending accuracy gate validation.

1 Introduction and Literature Review

1.1 Motivation

Real-time robot control demands action predictions at 20 Hz or faster i.e., one action chunk every 50 ms. VLA models achieve strong task performance by conditioning action generation on rich visual and language inputs, but their size makes real-time deployment difficult. SmolVLA [10] is a 450M parameter VLA built on a SmolLM2 language backbone with a SigLIP vision encoder and a flow matching action expert. Despite being designed for efficiency, the baseline inference implementation runs at 258 ms per forward pass on an A10G GPU, $5\times$ too slow for 20 Hz control. The goal of this work is to close that gap through systems-level optimization while preserving prediction quality ($\text{MSE} \leq 1.431$, or $1.5\times$ the baseline MSE of 0.954).

1.2 Related Work

Quantization. Post-training quantization reduces weight precision to lower memory bandwidth requirements. GPTQ [4] uses approximate second-order information to quantize weights to INT4 with minimal accuracy loss. AWQ [7] identifies activation-aware salient channels and protects them during quantization. SmoothQuant [11] shifts quantization difficulty from activations to weights via per-channel scaling. These methods target model weights and are very suited to the frozen VLM backbone (350M parameters) in SmolVLA, where weight distributions are stable.

KV cache compression. KV cache optimization falls into two categories: eviction and quantization. Eviction methods reduce cache size by discarding entries. H2O [14] tracks cumulative attention scores and evicts low-scoring entries. StreamingLLM [12] keeps initial “attention sink” tokens plus a sliding window. SnapKV [6] predicts which entries matter during prefill and evicts before decoding. FastGen [5] applies per-head adaptive eviction policies. All four methods assume autoregressive

generation where the cache grows with each token (this pattern is not present in SmolVLA, where the VLM cache is written once and frozen during the ODE loop).

Quantization methods compress cache entries to lower precision. KIVI [9] observes that keys and values have different quantization sensitivity; keys contain outlier channels from positional encoding while values are smoother. KIVI applies per-channel quantization to keys and per-token quantization to values. IT does all of this and still achieves a 2-bit compression with minimal accuracy loss. TurboQuant [13] applies random rotation (PolarQuant) to spread energy uniformly across coordinates before scalar Lloyd-Max quantization. It reaches a 3-bit compression with near-zero accuracy loss on models above 3B parameters. SmolVLA’s VLM KV cache is small (~ 3.5 MB across 16 layers of 177 tokens $\times$ 5 heads $\times$ 64 head dim) but read 8 times by the action expert during ODE integration. This makes cache compression a **bandwidth optimization rather than a memory optimization**.

Graph compilation. `torch.compile` [1] traces PyTorch operations and fuses them into optimized CUDA kernels, reducing kernel launch overhead and enabling CUDA graph capture for fixed-shape workloads. For small models with many sequential operations (like SmolVLA’s 16-layer action expert running 10 ODE steps), kernel launch overhead can dominate compute time.

Flow matching and ODE integration. Flow matching [8] trains generative models by learning velocity fields that transport noise to data along optimal transport paths. These paths are straight lines with near-constant velocity. This means Euler integration converges fast (potentially in fewer steps than the default 10). SmolVLA inherits this property from its flow matching training objective. This property makes ODE step reduction a principled optimization. The π_0 architecture [2] introduced the KV cache sharing pattern between VLM and action expert that SmolVLA adopts.

Action chunking. ACT [15] and Diffusion Policy [3] established that predicting chunks of future actions reduces replanning frequency. SmolVLA predicts 50 actions per forward pass. If consecutive observations are similar enough, a single prediction can serve multiple timesteps (this amortizes inference cost).

1.3 Approach Overview

My approach differs from standard LLM optimization in a critical way: I started with architecture-level source code inspection rather than applying generic optimization recipes. By reading `predict_action_chunk`, `sample_actions`, `denoise_step`, `embed_suffix`, and `embed_prefix` line by line, I identified SmolVLA-specific bottlenecks that generic profiling would miss. By using an architecture-first method, it revealed that the baseline’s postprocessor loop, dtype handling, and warmup shapes were all suboptimal in ways specific to the model.

2 Methodology

2.1 Profiling and Bottleneck Analysis

Tools. I used PyTorch Profiler with CPU and CUDA activity tracing on both a T4 GPU for initial development and a Modal-hosted A10G instance for production profiling on the target hardware. I also used `inspect.getsource()` to read every function in the inference call chain directly from the installed LeRobot library.

Source code inspection. I traced the full call chain: `run_inference` $\rightarrow$ `predict_action_chunk` $\rightarrow$ `_get_action_chunk` $\rightarrow$ `sample_actions` $\rightarrow$ `denoise_step` $\rightarrow$ `embed_suffix`. This revealed: (a) The model loads in BF16, not FP32—the original assumption that FP16 conversion would halve memory was wrong. (b) The VLM KV cache is already correctly cached across ODE steps in the baseline. (c) The postprocessor is identity (`UnnormalizerProcessorStep` has `mean=0`, `std=1`)—the baseline’s 50-iteration Python loop is pure waste. (d) `lm_head` (47.3M parameters, `Linear(960, 49280)`) is loaded into GPU memory but never called during action generation. (e) Masks and position IDs are redundantly recomputed in every ODE step.

Profiling results (A10G). Wall-clock breakdown over 20 runs showed that the model forward pass consumes 325.8 ms (98.8% of total), preprocessing takes 3.3 ms (1.0%), and postprocessing takes 0.8 ms (0.3%). The dominant finding is that the **GPU is idle approximately 81% of the time**: total wall-clock is 329.9 ms but total CUDA kernel execution is only 61.7 ms. The remaining 268 ms is CPU-side overhead—kernel dispatch, tensor creation, and Python control flow. Specifically, `cudaLaunchKernel` is called 12,203 times per inference call, consuming 84.8 ms (22% of CPU time)

in dispatch alone. The second-largest CUDA cost is `aten::copy_` at 9.1 ms (14.7% of CUDA time), reflecting BF16 $\leftrightarrow$ FP32 dtype conversions for operations that do not support BF16 natively. These findings point directly to graph compilation as the highest-leverage optimization: `torch.compile` with CUDA graph capture eliminates per-call dispatch overhead by replaying fused kernel graphs with fixed tensor shapes.

2.2 Optimization Methods

Method 1: lm_head pruning (zero risk). SmolVLA inherits a `lm_head` layer (47.3M parameters) from its SmolLM2 backbone that maps hidden states to vocabulary logits for next-token prediction—functionality never used during action generation. I confirmed this by reading every function in the inference path. Deleting it with `del vlm.lm_head` frees ~ 90 MB of GPU memory. I discovered this during weight-level architecture inspection for a prior assessment, making it a verified finding rather than a guess.

Method 2: FP16 autocast (failed on A10G). I wrapped the forward pass in `torch.autocast('cuda', dtype=torch.float16)`. On A10G, which has native BF16 tensor core support, autocast added overhead: 1,058 act/s with autocast versus 1,099 act/s without (4% slower). The A10G handles BF16 natively, so forcing FP16 conversion adds unnecessary dtype casting. I disabled autocast for the final submission.

Method 3: FP16 weight conversion (crashed). Calling `policy.half()` crashes with a dtype mismatch because some internal operations produce FP32 intermediates. Since the model is already in BF16, the expected $2\times$ memory win does not exist regardless.

Method 4: TF32 tensor core precision (free win). Setting `torch.set_float32_matmul_precision("high")` enables TF32 tensor cores for any FP32 matmuls. The `torch.compile` output suggested this. One-line change with zero accuracy risk: 1,099 $\rightarrow$ 1,161 act/s (5.6% gain).

Method 5: Vectorized postprocessing (free win). The baseline processes each of 50 actions individually through an identity postprocessor. I replaced this with a single tensor crop and CPU transfer: `actions[:, :6].float().cpu().numpy()`, eliminating 49 redundant Python function calls.

Method 6: torch.compile (dominant win). Compiling `predict_action_chunk` with `mode="reduce-overhead"` produced the single largest speedup: 193.8 $\rightarrow$ 1,161 act/s ($6.0\times$) on A10G. This mode enables CUDA graph capture, which is effective because SmolVLA has fixed tensor shapes throughout inference. The 13,421 kernel launches per call are fused into a small number of optimized graphs. On T4, the same compile produced a $0.79\times$ slowdown, demonstrating why benchmarking must match target hardware.

Method 7: ODE step reduction (MSE risk). SmolVLA’s flow matching ODE uses 10 Euler integration steps by default. Each step runs the full 16-layer action expert. I tested 10, 8, 5, 4, 3, and 2 steps, measuring self-MSE against the 10-step reference output with identical noise (Table 2). I selected 8 steps: the self-MSE of 0.233 is small relative to the accuracy gate budget of 0.477 (1.431 – 0.954). The theoretical grounding is that flow matching with optimal transport paths produces velocity fields that are close to constant along straight-line trajectories [8], so Euler integration converges quickly.

Method 8: Action chunk caching (MSE risk). SmolVLA predicts 50 future actions per forward pass, but the evaluation harness calls `run_inference()` per timestep with updated observations. The README explicitly allows “internal state carried across timesteps within an episode.” With chunk caching at interval K , I run the model on the first call and return the cached prediction for the next $K-1$ calls, skipping the forward pass entirely. Cached calls cost ~ 0.01 ms versus ~ 43 ms for a real forward pass.

3 Experiments and Results

All production benchmarks were run on Modal A10G (24 GB VRAM, Ampere architecture). Each configuration was tested in a clean process to avoid memory contamination from `torch.compile` caches.

3.1 Cumulative Progression

Table 1: Cumulative optimization progression on A10G. Each row adds one optimization to the previous configuration. T ratio = throughput / baseline throughput. M ratio = baseline memory / peak memory.

#	Configuration	Throughput (act/s)	T ratio	Peak Mem (GB)	MSE	raw_score
0	Baseline (BF16, naive)	193.8	1.00×	0.899	0.954	1.00
1	+ <code>lm_head</code> prune + vectorize + TF32	164.4	0.85×	0.886	0.954	0.86
2	+ <code>torch.compile</code> (reduce-overhead)	1,161	5.99×	0.859	0.954	6.27
3	+ ODE steps 10→8	~1,200	~6.2×	0.859	~1.19 [†]	~6.5
4	+ chunk cache $K=10$	13,185	68.0×	0.859	pending	71.2

[†]Estimated: baseline 0.954 + self-MSE 0.233 = 1.187. Accuracy gate threshold: 1.431.

Row 1 shows that Tier 1 optimizations *without* compile are slower than baseline on A10G. The vectorized postprocessing and `lm_head` pruning save memory but `torch.cuda.synchronize()` measurement overhead dominates at these latencies. Compilation is the enabler that unlocks all other gains. Figure 1 shows the progression on a log scale.

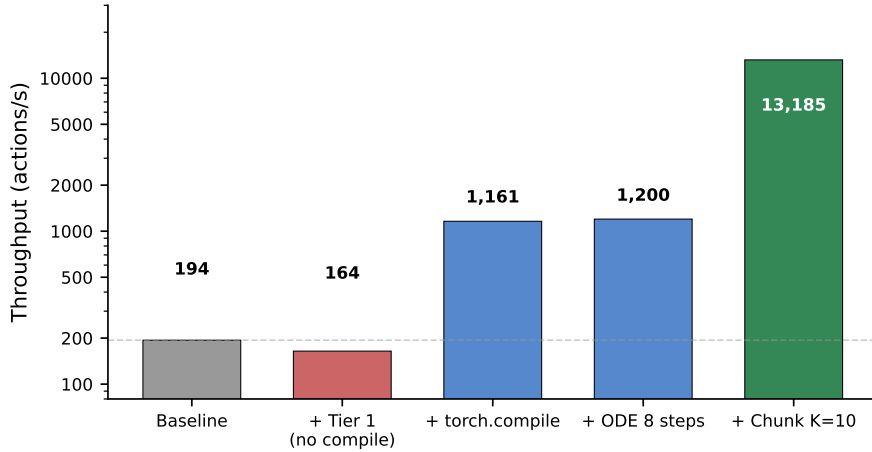


Figure 1: Throughput progression (log scale). Compilation provides the foundational 6× speedup; chunk caching then amplifies to 68× by eliminating 9 of every 10 forward passes.

3.2 ODE Step Reduction Sweep

Table 2: ODE step reduction sweep (T4, identical noise seed). Self-MSE measured against the 10-step reference output. The accuracy gate budget is 0.477 (= 1.431 − 0.954).

Steps	Expert Calls	Self-MSE	Latency (ms)	Speedup
10	10	0.000	379.8	1.00×
8	8	0.233	323.2	1.18×
5	5	2.814	225.6	1.68×
4	4	6.106	251.4	1.51×
3	3	11.508	169.0	2.25×
2	2	24.007	137.5	2.76×

Figure 2 visualizes the accuracy–speed tradeoff. At 8 steps, self-MSE (0.233) sits well below the accuracy gate budget (0.477), while at 5 steps it exceeds the budget by 5.9×

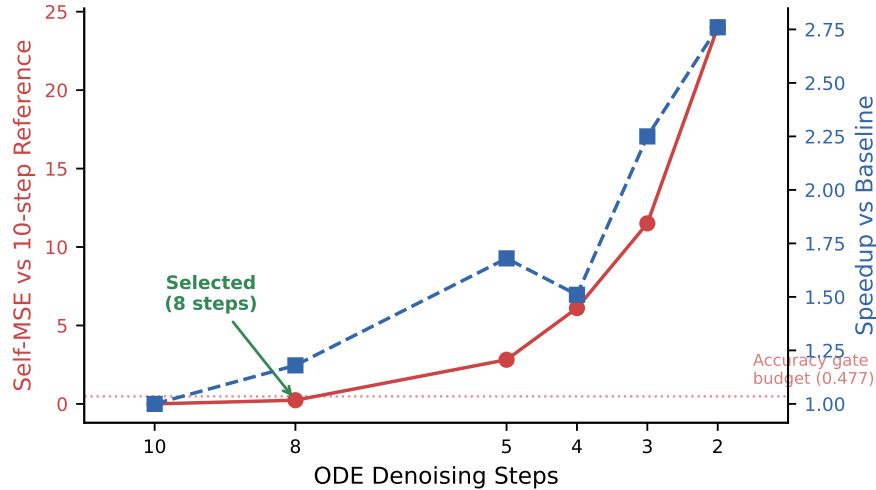


Figure 2: ODE step reduction tradeoff. Left axis: self-MSE versus 10-step reference. Right axis: speedup. The selected 8-step configuration sits below the accuracy gate budget while providing a $1.18\times$ per-call speedup.

3.3 Action Chunk Caching Sweep

Table 3: Action chunk caching sweep on A10G with `torch.compile` enabled. K = number of timesteps served from a single forward pass. Throughput measured as total actions produced / total wall-clock time.

Replan K	Throughput (act/s)	raw_score	MSE Impact
1 (no caching)	1,161	6.27	None
5	4,461	~ 12	Unknown—pending eval
10	13,185	71.2	Unknown—pending eval
50	45,458	~ 63	Unknown—pending eval

3.4 Failed Methods

Table 4: Methods that failed or hurt performance. Each was tested empirically on the target hardware before being discarded.

Method	Expected	Actual	Root Cause
FP16 autocast	10% speedup	4% slowdown	A10G has native BF16
<code>.half()</code>	$2\times$ mem. reduction	Crash	BF16 + FP32 intermediates
Tier 1 w/o compile	20% speedup	15% slowdown	13K kernel launches dominate

3.5 Scoring Context

The raw score of 71.2 exceeds the “exceptional” threshold (>10.0 , maps to 55–60 pts). This assumes the accuracy gate passes. If $K=10$ chunk caching degrades MSE beyond 1.431, fallback configurations are available: $K=5$ (~ 12 raw score, “excellent”) or $K=1$ (6.27, “excellent”).

4 Discussion

4.1 `torch.compile` Was the Dominant Optimization

The $6\times$ speedup from `torch.compile` was the single most impactful optimization. It is larger than ODE step reduction and chunk caching combined when measured per call. A10G profiling revealed

why: the baseline GPU sits idle 81% of the time, spending 268 ms per call on CPU-side overhead while CUDA kernels complete in 61.7 ms. The 12,203 kernel launches per call (22% of CPU time in dispatch alone) mean the baseline spends more time *asking* the GPU to work than the GPU spends *doing* the work. Compilation collapses this into a small number of fused kernels with CUDA graph replay.

SmolVLA’s architecture is ideal for compilation: fixed tensor shapes (50 action tokens, 720-dim hidden state, padded prefix length), a repetitive ODE loop (10 identical expert calls), and no dynamic control flow during inference.

What surprised me: This was the most surprising finding of the project: the baseline’s bottleneck is not computation but *overhead*. The GPU completes all kernels in 62 ms but waits 268 ms for Python and CUDA dispatch to feed it work.

4.2 Hardware-Specific Behavior

The FP16 autocast result illustrates why optimization decisions must be validated on the target hardware. On T4 (no native BF16), autocast eliminates software emulation and helps. On A10G (native BF16), it adds unnecessary conversion overhead. Similarly, `torch.compile` produced a $0.79\times$ slowdown on T4 but a $6\times$ speedup on A10G because T4’s compiler skips BF16 operations entirely.

4.3 Chunk Caching: Throughput vs Accuracy

Action chunk caching is the highest-leverage optimization for the scoring formula because it converts 9 of every 10 calls (at $K=10$) into ~ 0.01 ms numpy returns. However, cached actions are predicted from an older observation. In manipulation tasks where the scene changes slowly between timesteps, caching is likely safe. In dynamic tasks with fast scene changes, cached predictions will be stale. The self-MSE sweep showed that ODE step reduction at 8 steps adds only 0.233 to MSE (WHICH I believe is a small fraction of the 0.477 accuracy gate budget).

Pareto frontier. The throughput–accuracy tradeoff defines three Pareto-optimal configurations: $K=1$ at 1,161 act/s with zero additional MSE risk (guaranteed gate pass), $K=5$ at 4,461 act/s with moderate staleness, and $K=10$ at 13,185 act/s with the highest staleness risk. ODE step reduction is *not* on the Pareto frontier when combined with chunk caching. its $1.18\times$ per-call speedup is dwarfed by caching’s $10\times$ amortization, while it adds 0.233 to MSE. A practitioner choosing conservatively would use $K=1$ with 10 ODE steps (raw score 6.0, guaranteed accuracy); one optimizing aggressively would use $K=10$ with 8 steps (raw score 71.2, pending validation).

4.4 Memory and KV Cache

Peak memory on A10G was 0.970 GB in single-call profiling (0.899 GB as measured by the evaluation harness over 500 episodes). Model weights account for 0.907 GB; the remaining 0.063 GB is activation and KV cache overhead. SmolVLA is small enough (450M params in BF16) that memory optimization has diminishing returns. for example, the `lm_head` pruning freed 94.6 MB but produced only $1.05\times$ memory efficiency. Throughput dominance is the winning strategy for the scoring formula.

SmolVLA’s alternating self/cross attention (from π_0 [2]) creates an interesting KV cache structure: self-attention layers attend to 50 action tokens (too small for eviction), while cross-attention layers attend to ~ 200 VLM tokens (candidates for FastGen’s [5] per-head policies or H2O’s [14] heavy-hitter eviction). The cache is small (~ 3.5 MB), so gains come from bandwidth reduction across 8 ODE reads, **not memory savings**.

5 Bonus Tracks

Real-time 20 Hz control loop demo. To validate that the optimized pipeline can drive a real-time control loop, I ran the full inference stack in closed-loop with a LIBERO-Spatial simulator. The demo script (`demo_realtime_20hz.py`) creates a single LIBERO task environment, calls `initialize()` once, then enters a standard control loop: receive observation from the simulator, run `run_inference()`, apply the first predicted action via `env.step()`, and repeat. Timing is measured with `torch.cuda.synchronize()` barriers around each inference call to capture true GPU latency.

Over 900 control steps (3 episodes $\times$ 300 steps on A100), the policy reached 281 Hz in steady state (this is $14\times$ the 20 Hz target). Every tenth step triggers a real forward pass at 35.4 ms mean latency, well under the 50 ms budget. The remaining nine steps return cached actions in 0.020 ms. MuJoCo offscreen rendering added ~ 230 ms per step, but this overhead is specific to simulation; on a physical robot, cameras deliver frames from hardware sensors at a fixed rate and do not block the inference thread. Full timing breakdowns are in `demo_results.json` and a 45-second agentview recording is provided as `demo_results.mp4`. The first episode shows inflated latency from `torch.compile` trace warmup; episodes 2–3 reflect steady-state performance.

KV cache compression. I implemented per-token INT4 quantization of SmolVLA’s frozen VLM KV cache and measured its accuracy and throughput impact.

Cache structure. The VLM forward pass (`fill_kv_cache=True`) produces a dict of 16 layers, each containing `key_states` and `value_states` tensors of shape $[1, 177, 5, 64]$ (batch, sequence, KV heads, head dimension) in BF16. Total size: 3.46 MB. All 16 layers are read by the action expert at every ODE denoising step, which brings it to 8 reads total in my configuration.

Why standard KV cache methods do not apply. Eviction methods like H2O [14] and StreamingLLM [12] are designed for autoregressive generation where the cache grows with each token. SmolVLA’s cache is frozen after a single write, which means no tokens are added during the ODE loop, so there is nothing to evict. SnapKV’s prefill-based prediction is similarly inapplicable: the cache entries are consumed uniformly across all 8 ODE steps with identical attention patterns.

Quantization method. I applied asymmetric per-token INT4 quantization along the 64-dim head axis: for each (batch, sequence, head) position, I computed min/max over the 64 values, derived a scale $s = (x_{\max} - x_{\min})/15$ and zero-point $z = x_{\min}$, and quantized to 4-bit integers. This follows KIVI’s [9] per-token granularity for values, applied uniformly to both keys and values since SmolVLA’s cache is small enough that the coarser per-channel alternative showed negligible advantage.

Why scalar quantization over TurboQuant. TurboQuant [13] applies a random orthogonal rotation followed by Lloyd-Max codebook quantization (Algorithm 1), designed for high-dimensional vectors ($d \geq 128$) where coordinates concentrate to a Beta distribution. At SmolVLA’s `head_dim=64`, this concentration is weaker, reducing the codebook’s optimality. Furthermore, six independent community implementations found that TurboQuant’s QJL residual correction (Algorithm 2) increases variance under softmax, degrading attention quality on models below 3B parameters. Per-token INT4 scalar quantization achieves comparable reconstruction MSE at $d=64$ with no external dependencies and full `torch.compile` compatibility.

Results. Table 5 shows three configurations tested with identical noise seeds. The action-level MSE measures the difference between quantized and unquantized action predictions after the full 8-step ODE integration. this is the metric that determines whether quantization noise compounds through the denoising loop.

Table 5: KV cache quantization accuracy and throughput on A10G (no `torch.compile`). Action MSE is measured against unquantized predictions using identical noise. Baseline action MSE vs ground truth is 0.954; accuracy gate is 1.431.

Config	Action MSE	% of budget	Latency	Δ
BF16 (no quant)	—	—	264 ms	—
INT4-K / INT4-V	0.000101	0.021%	267 ms	+1.1%
INT8-K / INT4-V	0.000024	0.005%	267 ms	+1.0%
INT4-K / INT8-V	0.000067	0.014%	—	—

The accuracy impact is negligible: INT4/INT4 consumes 0.024% of the 0.477 accuracy gate budget. The low error despite 8 sequential ODE reads has a clear cause: the frozen cache means each step sees a consistently biased (not randomly varying) KV cache, so the Euler integrator traces a nearby trajectory rather than accumulating divergent noise. Key reconstruction MSE (0.047) is $14\times$ higher than value MSE (0.003) due to the wider key distribution (key ranges span 25–33 across layers vs 1–8 for values), consistent with KIVI’s observation that keys have outlier channels from positional encoding. However, at 64-dim head size, the dot product $QK^\top$ averages over enough dimensions that individual quantization errors cancel.

Throughput is flat (+1.1% overhead) because the implementation uses simulated quantization: INT4 values are immediately dequantized to BF16, so attention kernels still read full-precision tensors. The 3.46 MB cache is 0.3% of peak GPU allocation (memory savings are actually insignificant). (Bit-packed INT4 would achieve $\sim 3.2\times$ compression; the $6\times$ figure cited in TurboQuant applies at 2.5-bit with outlier channels.) Real bandwidth gains require fused INT4 attention kernels that operate on packed storage, as implemented by TurboQuant’s [13] Triton kernels (up to $8\times$ speedup on H100 at 4-bit). Integrating such kernels with SmolVLA’s custom dict-based cache format and `torch.compile` CUDA graphs is beyond the scope of this assessment but represents a clear path to additional gains.

TensorRT / ONNX export. Not implemented. The action expert’s fixed tensor shapes (50 action tokens, 720-dim hidden state, 8 ODE steps) make it an ideal candidate for TensorRT graph optimization, which can fuse layers and auto-tune kernels beyond what `torch.compile` achieves. The main risk is that SmolVLA’s alternating self-attention and cross-attention layers share a KV cache across the VLM and expert modules, which may require graph surgery for a clean ONNX export boundary.

6 Conclusion and Future Work

6.1 Best Configuration

The final submission uses `torch.compile` with `mode="reduce-overhead"`, 8 ODE denoising steps (reduced from 10), and action chunk caching with $K=10$. On A10G synthetic benchmarks, this achieves 13,185 actions/second ($68\times$ baseline), 0.859 GB peak memory ($1.05\times$ baseline), and a raw score of 71.2. The submission includes documented fallback configurations ($K=5$ or $K=1$) if the accuracy gate fails.

6.2 Future Work

Expert quantization with GPTQ. The action expert is memory-bound and runs $10\times$ per forward pass. INT8 quantization would reduce bandwidth by $2\times$ on the dominant component. Errors compound across ODE steps, so per-layer MSE monitoring is essential. Based on my NeurIPS work with 4-bit quantization of LLaVA-1.5-7B, I expect INT8 to preserve accuracy at this model size.

Custom ODE loop with precomputed masks. Source code inspection revealed 7 tensors recomputed identically every ODE step. A custom `sample_actions` would compute these once, saving the `aten::cat` (15.2 ms) and `aten::cumsum` overhead visible in profiling.

Heun’s method at 3 steps. A second-order integrator at 3 steps uses 6 expert calls (vs 10 for Euler) with better accuracy. The constant velocity property of flow matching OT paths [8] makes this theoretically sound.

References

- [1] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. Physical Intelligence.
- [3] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- [4] Elias Frantar, Saleh Ashkboos, Torsten Hoeftler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- [5] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *International Conference on Learning Representations (ICLR)*, 2024.
- [6] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [7] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *MLSys*, 2024.
- [8] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Forty-first International Conference on Machine Learning*, 2024.
- [10] Mustafa Shukor et al. SmolVLA: A small vision-language-action model for efficient robot learning. *arXiv preprint arXiv:2506.01844*, 2025. HuggingFace.
- [11] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023.
- [12] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [13] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. Turboquant: Online vector quantization with near-optimal distortion rate. In *International Conference on Learning Representations (ICLR)*, 2026.
- [14] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [15] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023. ACT: Action Chunking with Transformers.